National University of Computer & Emerging Sciences

Lecture 19
Joinable and Detached Threads

The Pthreads API

- The subroutines which comprise the Pthreads API can be informally grouped into four major groups:

- **Thread management:** Routines that work directly on threads - creating, detaching, joining, etc. They also include functions to set/query thread attributes (joinable, scheduling etc.)
- **Mutexes, Condition variables, Synchronization**

Thread Creation

- Initially, your main() program comprises a single, default thread. All other threads must be explicitly created by the programmer.

- pthread_create creates a new thread and makes it executable. This routine can be called any number of times from anywhere within your code.

Thread Creation

```
pthread_create( pthread_t *threadid
   ,const pthread_attr_t *attr, void
  *(*start_routine)(void *),void *arg);
```

- **threadid**
  - The routine returns the new thread ID via the threadid
  - The caller can use this thread ID to perform various operations
  - This ID should be checked to ensure that the thread was successfully created.
- **attr**
  - used to set thread attributes.
  - NULL for the default values.
- **start_routine**
  - The C routine that the thread will execute once it is created.
- **arg**
  - Arguments are passed to *start_routine* via *arg*.
  - Arguments must be passed by reference as pointers
  - These pointers must be cast as pointers of type void.

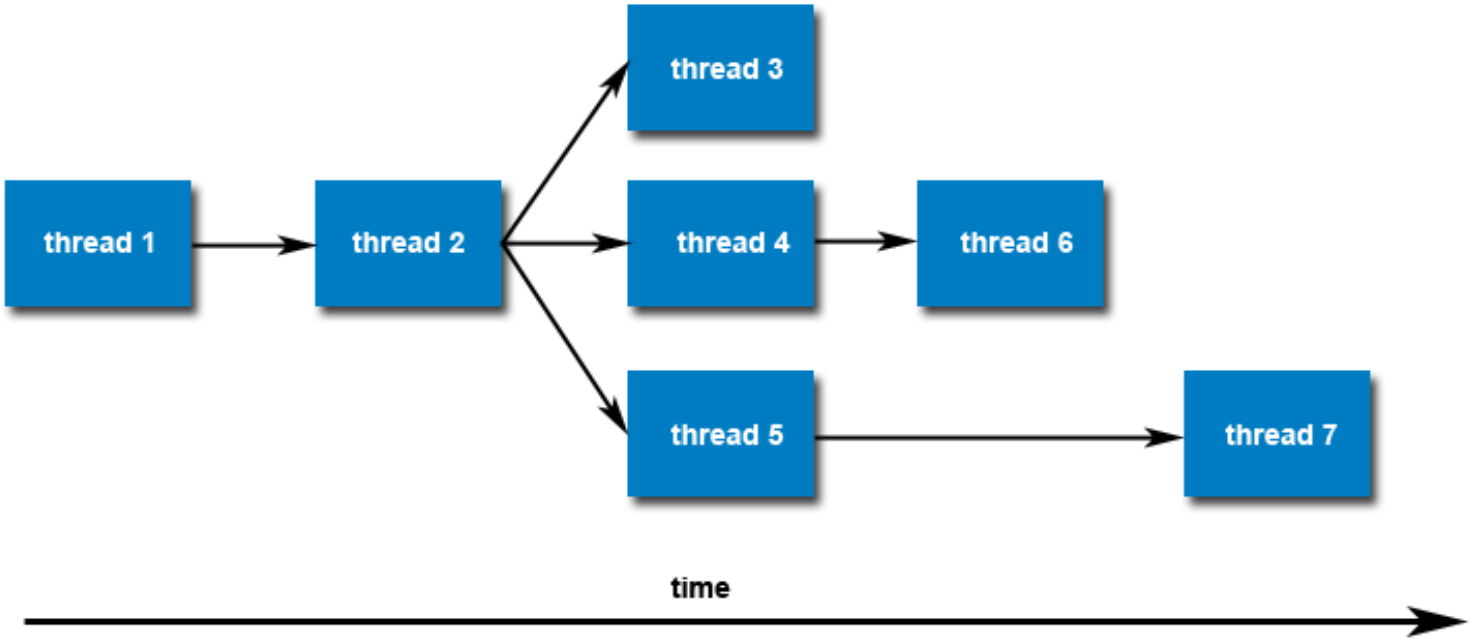
```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

void * PrintHello(void *threadid)
{   printf("\n%d: Hello World!\n", threadid);
    pthread_exit(NULL);
}
int main()
{
   pthread_t threads[NUM_THREADS];
   int rc, t;
   for(t=0;t < NUM_THREADS;t++)
  {
    printf("Creating thread %d\n", t);
    rc = pthread_create(&threads[t], NULL, PrintHello, (void *)&t);
    if (rc)
    {
       printf("ERROR; return code from pthread_create() is %d\n", rc);
       exit(-1);
    }
 }
pthread_exit(NULL); }
```

```
In main: creating thread 0
In main: creating thread 1
Hello World! It's me, thread #0!
In main: creating thread 2
Hello World! It's me, thread #1!
Hello World! It's me, thread #2!
In main: creating thread 3
In main: creating thread 4
Hello World! It's me, thread #3!
Hello World! It's me, thread #4!
```

Thread Creation

- Once created, threads are peers, and may create other threads. There is no implied hierarchy or dependency between threads.



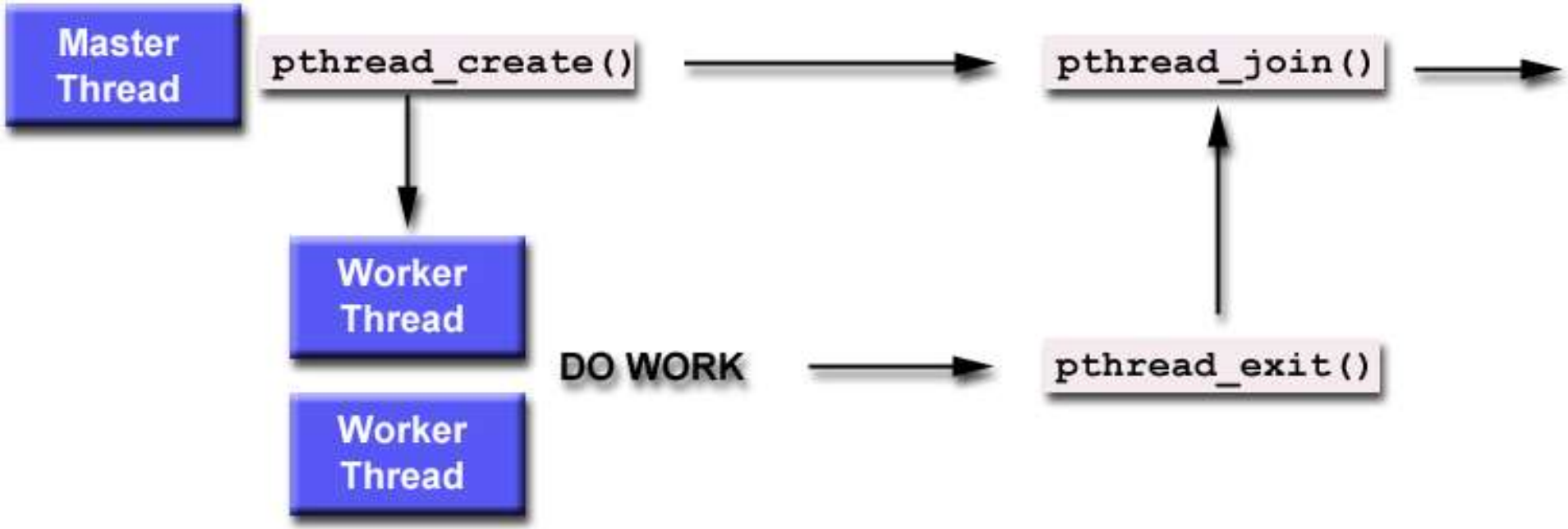
Attribute Object

- When an attribute object is not specified, it is NULL, and the *default* thread is created with the following attributes:
  - It is non-detached
  - It has a a default stack and stack size
  - It inherits the parent's priority

Thread Suspension and Termination

- Similar to UNIX processes, threads have the equivalent of the wait() and exit() system calls
- pthread_join()
  - Used to block threads
- pthread_exit()
  - Used to terminate threads
- To instruct a thread to block and wait for a thread to complete, use the **pthread_join()** function.
- This is also the mechanism used to get a return value from a thread
- Any thread can call join on (and hence wait for) any other thread.

Thread Suspension and Termination



Joinable or Not?

- When a thread is created, one of its attributes defines whether it is joinable or detached. Only threads that are created as joinable can be joined. If a thread is created as detached, it can never be joined.

Detached State

- Each thread can be either joinable or detached.
- Detached: on termination all thread resources are released by the OS.
- A detached thread cannot be joined.
- No way to get at the return value of the thread.

Joining thread

- **Joinable:** on thread termination the thread ID and exit status are saved by the OS.
- Joining a thread means waiting for a thread
- **pthread_join(threadid, status)**
  - "Joining" is one way to accomplish synchronization between threads.
  - subroutine blocks the calling thread until the specified *threadid* thread terminates.
    - The programmer is able to obtain the target thread's termination return status (if specified) in the *status* parameter.
- It is impossible to join a detached thread

Joining thread

- Multiple threads cannot wait for the same thread to terminate.
- If they try to, one thread returns successfully
- The others fail with an error of ESRCH
- After pthread_join() returns, any stack storage associated with the thread can be reclaimed by the application.
- Threads which have exited but have not been joined are equivalent to zombie processes.
- Their resources cannot be fully recovered.

- To explicitly create a thread as joinable or detached, the attr argument in the pthread_create() routine is used. The typical 4 step process is:
- Declare a pthread attribute variable of the pthread_attr_t data type
- Initialize the attribute variable with pthread_attr_init()
- Set the attribute detached status with pthread_attr_setdetachstate()
- When done, free library resources used by the attribute with pthread_attr_destroy()

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define NUM_THREADS          4

void *BusyWork(void *t)
{
   int i;
   long tid;
   double result=0.0;
   tid = (long)t;
   printf("Thread %ld starting...\n",tid);
   for (i=0; i<1000000; i++)
   {
      result = result + sin(i) * tan(i);
   }
   printf("Thread %ld done. Result = %e\n",tid, result);
   pthread_exit((void*) t);
}
```

```
int main (int argc, char *argv[])
{
   pthread_t thread[NUM_THREADS];
   pthread_attr_t attr;
   int rc;
   long t;
   void *status;

   /* Initialize and set thread detached attribute */
   pthread_attr_init(&attr);
   pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);

   for(t=0; t<NUM_THREADS; t++) {
      printf("Main: creating thread %ld\n", t);
      rc = pthread_create(&thread[t], &attr, BusyWork, (void *)t);
      if (rc) {
         printf("ERROR; return code from pthread_create()
                is %d\n", rc);
         exit(-1);
         }
   }
```

```
/* Free attribute and wait for the other threads */
   pthread_attr_destroy(&attr);
   for(t=0; t<NUM_THREADS; t++) {
      rc = pthread_join(thread[t], &status);
      if (rc) {
         printf("ERROR; return code from pthread_join()
                is %d\n", rc);
         exit(-1);
         }
      printf("Main: completed join with thread %ld having a status
            of %ld\n",t,(long)status);
      }

printf("Main: program completed. Exiting.\n");
pthread_exit(NULL);
}
```

```
Main: creating thread 0
Main: creating thread 1
Thread 0 starting...
Main: creating thread 2
Thread 1 starting...
Main: creating thread 3
Thread 2 starting...
Thread 3 starting...
Thread 1 done. Result = -3.153838e+06
Thread 0 done. Result = -3.153838e+06
Main: completed join with thread 0 having a status of 0
Main: completed join with thread 1 having a status of 1
Thread 3 done. Result = -3.153838e+06
Thread 2 done. Result = -3.153838e+06
Main: completed join with thread 2 having a status of 2
Main: completed join with thread 3 having a status of 3
Main: program completed. Exiting.
```

pthread_detach()

- By default threads are created joinable.
- Instead of waiting for a child, a parent thread can specify that
  - it does not require a return value
  - or any explicit synchronization with that thread.
- To do this, the parent thread uses the **pthread_detach()** function.
- After this call, there is no thread waiting for the child – it executes independently until termination.
- To avoid memory leaks a thread should
  - either be *joined*
  - Or detached by a call to **pthread_detach()**
- int pthread_detach(pthread_t tid);
  - Returns 0 on OK, nonzero on error
- Threads can detach themselves by calling *pthread_detach* with an argument of *pthread_self*

```
#include<stdio.h>
#include<pthread.h>
void run() {
   pthread_exit(0);
}

int main () {
   pthread_t thread;
   int rc;
   long count = 0;
   while(1) {
      if(rc = pthread_create(&thread, 0, run, 0) ) {
         printf("ERROR, rc is %d, so far %ld threads created\n", rc, count);
         perror("Fail:");
         return -1;
      }
      count++;
   }
   return 0;
}
```